Design and Analysis of Algorithms

ASSIGNMENT 1

CO1: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration:1week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Design the Algorithm 5 Marks

Design and propose **at least three meaningfully different algorithmic strategies** for locating a given registration number.

The three approaches must represent **distinct algorithmic strategies**. Iterative and recursive implementations of the **same underlying algorithm will not be considered as two different approaches**.

Your proposed approaches must include:

- At least **one iterative approach**
- At least **one recursive approach**
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to **design and select the approaches independently**. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

TASK 2 – Identify the Basic Operation 2 Marks

For each of the proposed algorithms, identify the **basic operation** that predominantly determines its execution time.

Discuss:

- What the basic operation is
- Where it occurs in the algorithm
- Why it is appropriate to use this operation for efficiency analysis

- How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to **relate the selected operation to the behaviour of the algorithm**.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

- The input size (n)
- The number of times the basic operation executes
- How the operation count changes with (n)
- The mathematical expression representing the running time
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the **mathematical reasoning leading to the complexity result**.

Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour4 Marks

Analyse the performance of each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the **best case**

- What input condition produces the **average case**
- What input condition produces the **worst case**
- How the number of operations changes in each case
- Why these differences occur

Do not simply provide a table containing complexity values. You are expected to **explain the conditions that lead to each case**.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis 3 Marks

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

- Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant
- How each algorithm behaves as (n) becomes very large
- How the growth rates of the proposed approaches differ
- What the growth behaviour means when the dataset increases from **1,000 to 1,000,000+ records**

You are expected to **interpret the asymptotic behaviour in the context of the given application**, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

TASK 6 – Formulate the Recurrence Relation 3 Marks

For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm.

Clearly identify:

- The input size of the original problem
- The size of the recursive subproblem
- The number of recursive calls
- The work performed outside the recursive call
- The base case
- How these components lead to the recurrence relation

The recurrence must be **derived from your algorithm**, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation 4 Marks

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method.

You may use:

- Substitution method
- Iteration method
- Master Theorem
- Another appropriate analytical technique

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to **show the analytical process**, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms 4 Marks

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

- All algorithms produce the correct result
- The same or equivalent datasets are used for comparison
- Testing conditions are kept reasonably consistent
- Appropriate test cases are included
- The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is **not merely to demonstrate working code**, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

TASK 9 – Record Experimental Performance 3 Marks

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

- Execution time
- Number of comparisons
- Number of key operations
- Input size
- Other relevant performance measures, where justified

Present the results using appropriate:

- Tables
- Graphs
- Performance plots

Ensure that the presentation allows the different approaches to be easily compared.

Evidence Expected

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

TASK 10 – Compare Theory with Experimental Behaviour 4 Marks

Compare your theoretical analysis with your experimental results.

Discuss:

- Whether the observed growth agrees with the theoretical complexity
- Which algorithm scales better as (n) increases
- Any differences between theoretical prediction and actual execution
- Possible reasons for the differences

You are expected to **interpret the results**, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

- Which approach you recommend
- Why it is more appropriate than the alternatives
- Its theoretical efficiency
- Its experimentally observed performance
- Any preprocessing cost involved

- Relevant trade-offs or limitations
- Whether the solution remains practical as the database grows

Your recommendation must be based on **evidence obtained from your own analysis and experimentation**.

Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**.

However, new student records are added only **once per day**.

Re-evaluate your original design in response to these changed operating conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions are affected
- Why the frequency of searches is now important
- Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

- Its expected behaviour under 50,000 searches per hour
- Whether its preprocessing cost is acceptable under the new conditions
- Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

- Describe the revised algorithmic strategy
- Explain what has changed
- Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

- Mathematical complexity analysis

- Experimental results, where appropriate
- Search frequency
- Preprocessing cost
- Dataset size
- Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

Design, Analyse and Validate an Efficient Search Strategy

Large-Scale University Student Record System

The assignment requires **three meaningfully different search strategies**, mathematical analysis, recurrence formulation/solution, implementation/testing, comparison, and a redesign for **50,000 searches/hour on 1,000,000 records**. Design and Analysis of Algorith…

1. Problem Understanding

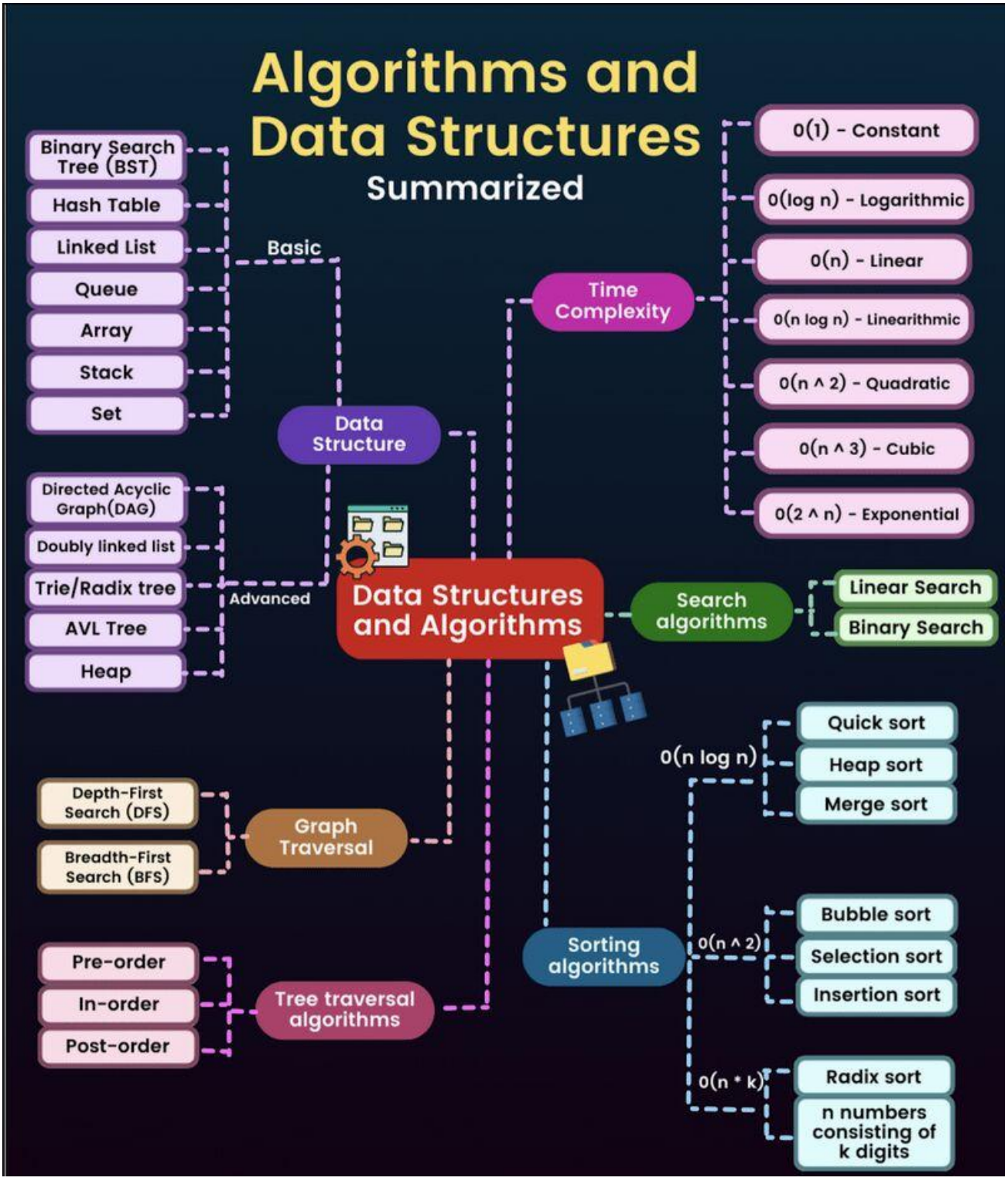
The university stores student records using a unique **Registration Number**. The database may grow from **1,000 to 1,000,000+ records**, and the system must efficiently determine whether a registration number exists and retrieve the corresponding record. Preprocessing such as sorting or indexing must also be included in the analysis. Design and Analysis of Algorith…

Proposed three strategies

Strategy	Design	Main idea	Expected search
1. Hash Table	Iterative	Hash registration number → table position	Θ(1) average
2. Binary Search	Iterative	Repeatedly divide sorted data into halves	Θ(log n)
3. B-tree Search	Recursive	Traverse a balanced multiway index	Θ(log n)

These are genuinely different because they use different **data structures and search strategies**, rather than merely changing iterative code into recursive code.

2. Concept Map


Algorithms and Data Structures
Summarized
Binary Search Tree (BST)
Hash Table
Linked List
Queue
Array
Stack
Set
Basic
Data Structure
Directed Acyclic Graph(DAG)
Doubly linked list
Trie/Radix tree
AVL Tree
Heap
Advanced
Time Complexity
O(1) - Constant
O(log n) - Logarithmic
O(n) - Linear
O(n log n) - Linearithmic
O(n ^ 2) - Quadratic
O(n ^ 3) - Cubic
O(2 ^ n) - Exponential
Data Structures and Algorithms
Search algorithms
Linear Search
Binary Search
Sorting algorithms
O(n log n)
Quick sort
Heap sort
Merge sort
O(n ^ 2)
Bubble sort
Selection sort
Insertion sort
O(n * k)
Radix sort
n numbers consisting of k digits
Graph Traversal
Depth-First Search (DFS)
Breadth-First Search (BFS)
Tree traversal algorithms
Pre-order
In-order
Post-order

Innovative idea: Use a **"Search Highway" concept** in your presentation:

- Hash table = **express lane** → fastest average lookup.
- Binary search = **divide-and-conquer lane** → predictable logarithmic search.
- B-tree = **multi-level highway interchange** → scalable for huge dynamic databases.

TASK 1 – Design the Algorithms

The assignment specifically requires at least three distinct strategies, including at least one iterative and one recursive approach. Design and Analysis of Algorith…

Algorithm 1: Hash Table Search — Iterative

Basic idea

A hash function converts the registration number into an array index.

Registration Number

↓

Hash Function

↓

Table Index

↓

Student Record

For example:

Reg No = 20261025

index = hash(20261025)

The system directly accesses the corresponding location.

Preprocessing

Build a hash table from all student records.

For n records:

Pseudocode

```
HASH_SEARCH(table, regNo)

    index ← HASH(regNo)

    if table[index] is empty
        return NOT FOUND

    if table[index].regNo = regNo
        return table[index]

    else
        search collision chain

    return NOT FOUND
```

Collision Handling in a Hash Table

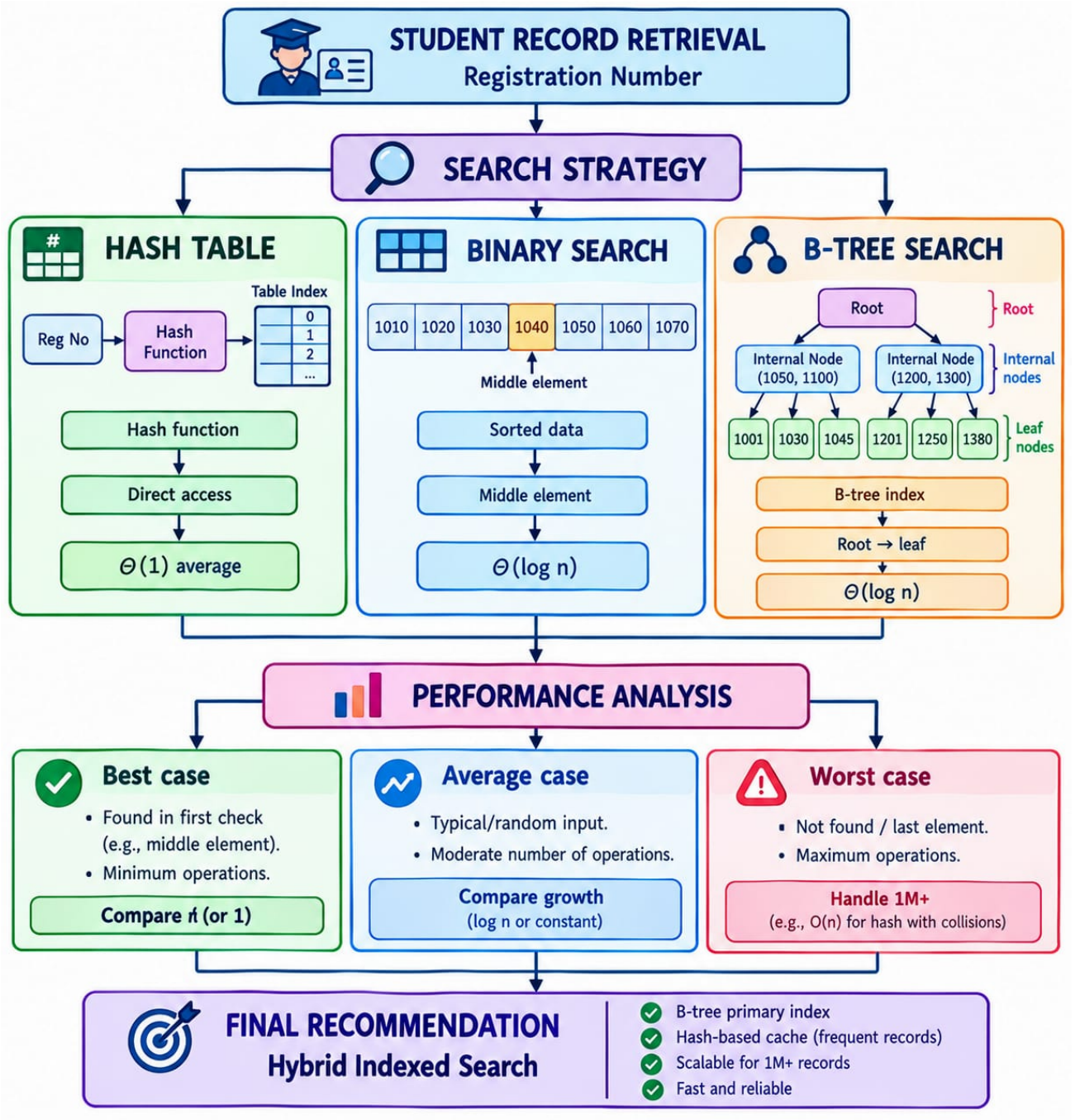
A collision occurs when two different student registration numbers produce the same hash index. Collisions can be handled using techniques such as separate chaining or open addressing. In separate chaining, multiple records are stored in a list at the same index. In open addressing, another available position is searched for storing the record. The average search time is $\Theta(1)$ when the hash function distributes keys uniformly and the load factor is controlled. However, the worst-case search time is $O(n)$, where n is the number of records.

Why choose it?

It is ideal when:

- registration numbers are unique,
- searches are frequent,

- fast average lookup is important,
- sufficient memory is available.



Algorithm 2: Binary Search — Iterative

Basic idea

First sort the records according to registration number.

Then compare the target with the middle record.

Sorted Records

1001 1015 1022 1034 1045 1060 1078

↑

middle

If target < middle:

Search left half

If target > middle:

Search right half

Therefore, approximately half of the remaining records are eliminated during every iteration.

Preprocessing

Sorting:

Pseudocode

```
BINARY_SEARCH(A, regNo)

    low ← 0
    high ← n - 1

    while low ≤ high

        mid ← floor((low + high)/2)

        if A[mid] = regNo
```

```
        return A[mid]

    else if regNo < A[mid].regNo
        high ← mid - 1

    else
        low ← mid + 1

  return NOT FOUND
```

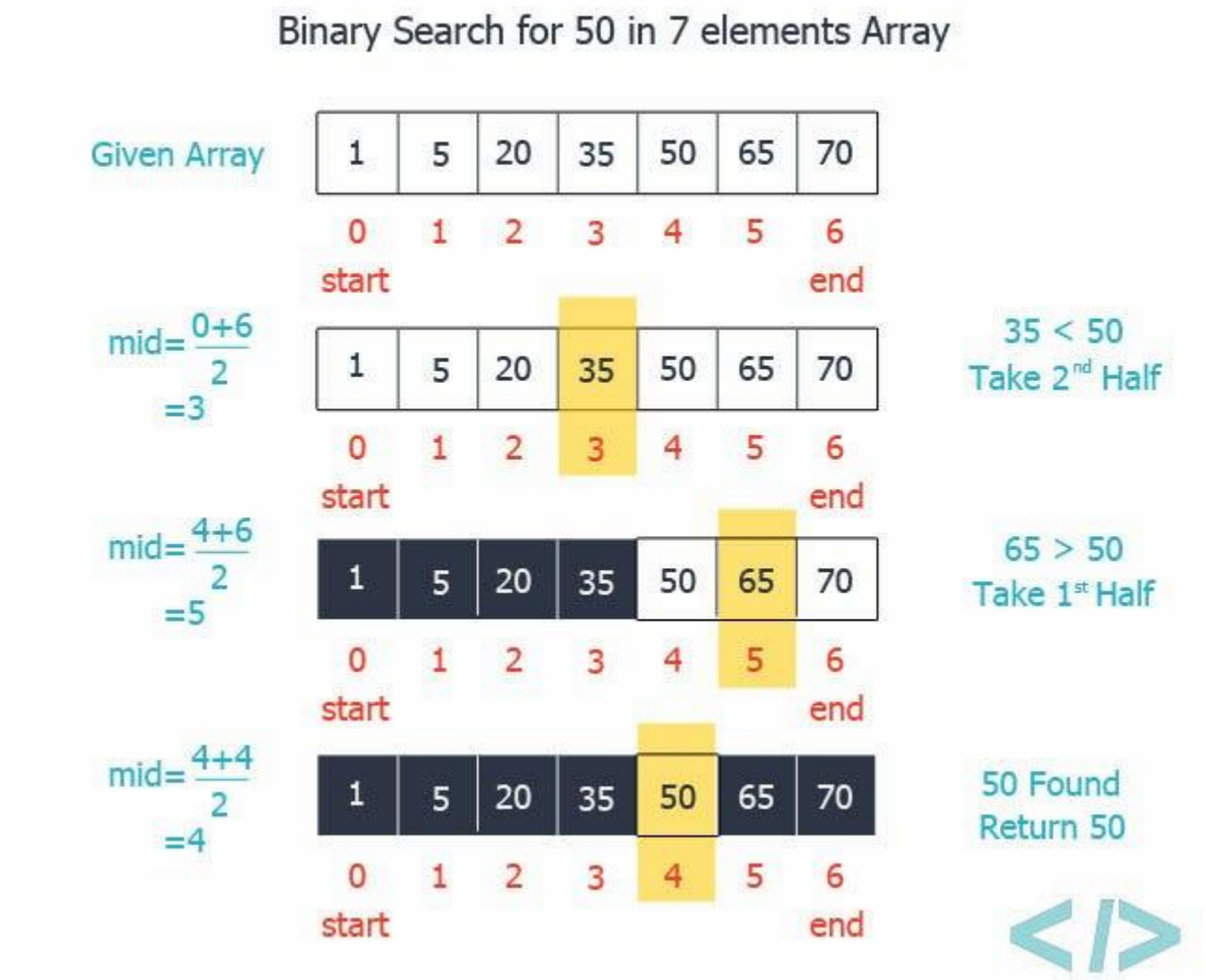
Prerequisite for Binary Search

Binary search requires the student registration records to be sorted by registration number before searching. It repeatedly compares the target registration number with the middle element and eliminates half of the remaining search space. Its worst-case search time is O(log n). If the records are not sorted, they must be sorted first, and this preprocessing cost should be considered separately.

Why choose it?

Binary search is:

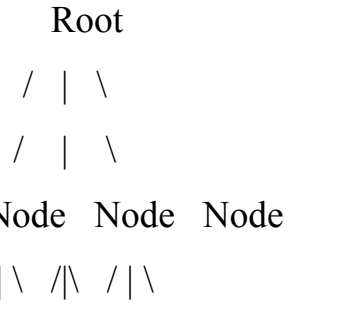
- simple,
- memory efficient,
- deterministic,
- excellent for a sorted static dataset.



Algorithm 3: Recursive B-tree Search

A B-tree is especially suitable for very large databases because it keeps data organized in a balanced multi-level index.

Basic idea



Instead of examining millions of records, the algorithm moves through only a small number of levels.

Preprocessing

Construct the B-tree:

For a balanced B-tree, searching requires approximately:

node levels.

Recursive pseudocode

BTREE_SEARCH(node, regNo)

```
if node = NULL
    return NOT FOUND

find the first key ≥ regNo

if key = regNo
    return corresponding record

if node is a leaf
    return NOT FOUND
```

child ← appropriate child

return BTREE_SEARCH(child, regNo)

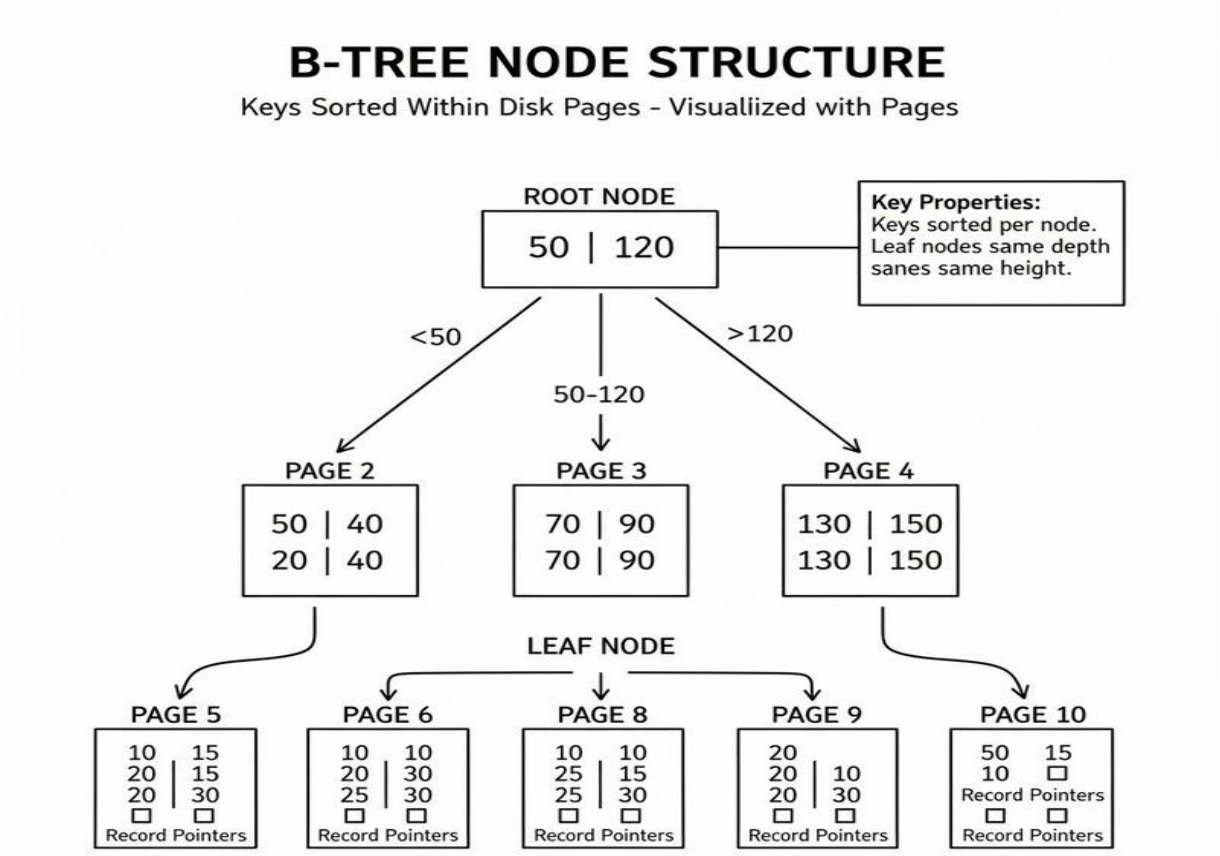
B-Tree Search Complexity

A B-tree is a balanced search tree that stores multiple keys in each node. Searching follows the appropriate child pointers until the target key is found or the search reaches a leaf. For a balanced B-tree, the search time is O(log n) when measured in tree levels, assuming the tree has a fixed branching factor. B-trees are particularly useful for databases and large datasets because they reduce the number of node accesses.

Why choose it?

B-tree search is attractive because it supports:

- very large datasets,
- balanced searching,
- dynamic insertion,
- efficient indexed access.



TASK 2 – Basic Operation

The assignment asks for the dominant operation, where it occurs, and how its frequency changes with n. Design and Analysis of Algorith…

Algorithm	**Basic operation**	**Location**	**Effect as n increases**
Hash	Hash calculation + key comparison	Table lookup	Approximately constant average
Binary	Key comparison	Middle element	Number of comparisons grows logarithmically
B-tree	Key comparison/node traversal	Each tree level	Number of levels grows logarithmically

Hash table

The main operation is:

With a good hash function, only a small number of locations are examined.

Binary search

The key comparison happens at:

Each comparison eliminates approximately half the remaining data.

B-tree

Key comparison determines which child should be followed.

Because the tree remains balanced, only a logarithmic number of levels are visited.

TASK 3 – Mathematical Time Complexity

The assignment requires showing the mathematical reasoning instead of merely writing Big-O notation. Design and Analysis of Algorith…

A. Hash Table

Assume average constant-time lookup.

For one search:

Therefore:

Hence:

Preprocessing:

Therefore:

The important point is that the **one-time construction cost is linear**, while repeated

searches are very fast.

B. Binary Search

Initially:

records are present.

After one comparison:

After two:

After k comparisons:

Search stops when:

Therefore:

Taking log:

Hence:

Including sorting:

Therefore:

C. B-tree Search

For a balanced B-tree, each level reduces the remaining search space substantially.

Let the height be h.

For branching factor b:

Taking logarithm:

Therefore:

Index construction:

Thus:

Requirement	**Hash**	**Binary**	**B-tree**
Fast direct lookup	⭐⭐⭐	⭐⭐	⭐⭐
Large database	⭐⭐⭐	⭐⭐	⭐⭐⭐
Dynamic records	⭐⭐	⭐	⭐⭐⭐
Ordered search	⭐	⭐⭐⭐	⭐⭐⭐
Low extra memory	⭐	⭐⭐⭐	⭐⭐
Frequent searches	⭐⭐⭐	⭐⭐	⭐⭐⭐
Predictable performance	⭐⭐	⭐⭐⭐	⭐⭐⭐

Preprocessing Cost

Preprocessing time should be measured separately from search time. For binary search, sorting n records typically takes O(n log n) time using an efficient comparison-based sorting algorithm. For a hash table, building the table takes expected O(n) time with a suitable hash function. Building a B-tree by inserting n records takes O(n log n) time under standard fixed-order assumptions. These construction costs are generally incurred before searching, although updates may require additional work later.

TASK 4 – Best, Average and Worst Cases

The assignment specifically requires the **conditions causing** each case, not merely a table of complexities. Design and Analysis of Algorith…

Algorithm	**Best**	**Average**	**Worst**
Hash table	Θ(1)	Θ(1)	O(n)
Binary search	Θ(1)	Θ(log n)	Θ(log n)
B-tree	Θ(1) / Θ(log n)*	Θ(log n)	Θ(log n)

Best-Case Time Complexity

The best-case time complexity of binary search is Θ(1). This occurs when the target registration number is equal to the middle element in the first comparison. In this situation, the algorithm finds the target immediately without performing further search steps.

Hash table

Best case: target maps directly to an empty-collision location.

Average case: good hash distribution gives approximately constant lookup.

Worst case: many keys collide.

If all n records form one collision chain:

Therefore:

Binary search

Best case: target is exactly the first middle element.

Worst case: target is absent or found near the final division.

B-tree

Best case occurs when the target is found near the root.

Worst case occurs when the algorithm travels from root to a leaf.

Because the tree remains balanced:

TASK 5 – Asymptotic Analysis

The assignment requires interpretation of growth as the dataset increases. Design and Analysis of Algorith…

Growth comparison

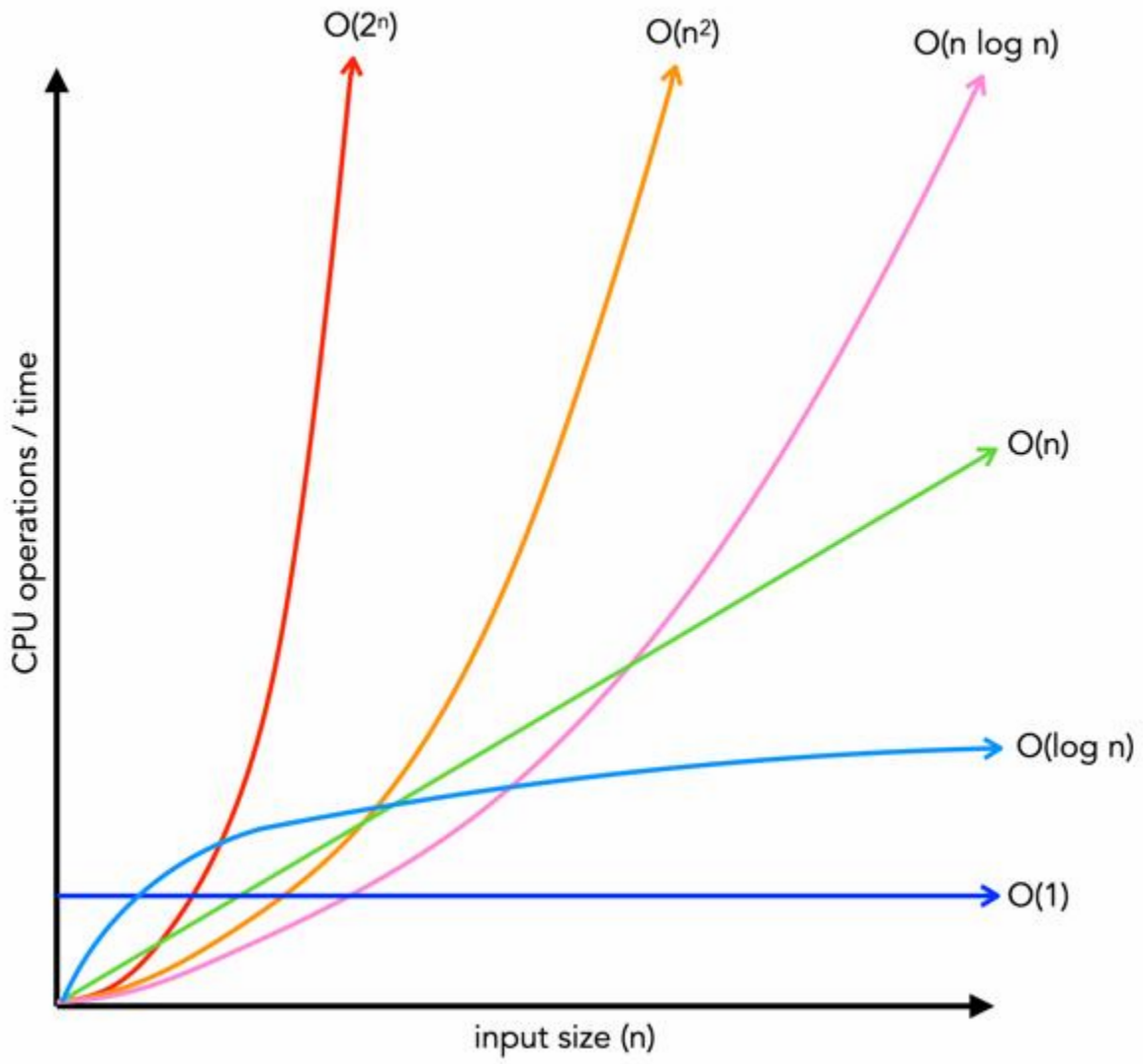
For example, when:

Binary search requires roughly:

comparisons.

So instead of potentially checking one million records, only around **20 levels of decisions** are needed.

This demonstrates why logarithmic algorithms remain practical for million-record datasets.



TASK 6 – Recurrence Relation

For the recursive B-tree search, let:

- n = size of the current problem,
- one recursive call is made,
- the next subproblem contains approximately a fraction of the current node's search space,
- constant work is performed to compare keys and select the child.

For a simplified binary recursive representation:

Base case:

Therefore:

This recurrence directly represents:

Current node

↓

Compare keys

↓

Select ONE child

↓

Search smaller subproblem

Recurrence Relation for Recursive B-Tree Search

Let h be the height of the B-tree. At each recursive step, the algorithm examines the current node and follows at most one child pointer. Therefore, the recurrence can be written as:

$T(h) = T(h - 1) + O(1)$

with the base case $T(0) = O(1)$, assuming a fixed maximum number of keys per node and constant-time node processing.

Expanding the recurrence gives $T(h) = O(h)$. Since a balanced B-tree with a fixed branching factor has height $O(\log n)$, the search complexity is $O(\log n)$.

TASK 7 – Solve the Recurrence

We use the **iteration/substitution method** because the recurrence repeatedly reduces the problem size by half.

Given:

Expand once:

Expand again:

After k expansions:

The base case occurs when:

Therefore:

Substitute:

Therefore:

Interpretation

As the database grows from 1,000 to 1,000,000 records, the search time does **not** grow proportionally to the number of records. It grows logarithmically.

Recurrence Relation for Binary Search

The recurrence relation for binary search can be expressed as:

$T(n) = T(n/2) + c$

where T(n) represents the search time for n elements and c is the constant time required for one comparison and related operations. At each step, binary search reduces the remaining search space by approximately half. Solving this recurrence gives $T(n) = \Theta(\log n)$.

This recurrence is a simplified model for binary search. A B-tree has multiple child pointers per internal node, so its analysis depends on the tree's branching factor and height rather than the binary-search recurrence alone.

TASK 8 – Implementation and Testing

The assignment requires testing approximately:

records. Design and Analysis of Algorith…

Test cases

For every dataset, use:

1. **Existing record near beginning**
2. **Existing record near middle**
3. **Existing record near end**
4. **Non-existing registration number**
5. Multiple repeated searches

Example:

Dataset	Search Key
1,000	Existing
1,000	Existing
1,000	Not found
10,000	Existing
10,000	Not found
100,000	Existing
100,000	Not found
1,000,000	Existing
1,000,000	Not found

Keep the dataset equivalent for all three algorithms so that the comparison is fair.

TASK 9 – Experimental Performance

The assignment asks for execution time, comparisons/key operations and input size, presented using tables or graphs. Design and Analysis of Algorith…

Use a table like this **after running your program**:

Records	**Hash comparisons**	**Binary comparisons**	**B-tree comparisons**
1,000	___	___	___
10,000	___	___	___
100,000	___	___	___
1,000,000	___	___	___

And:

Records	**Hash time**	**Binary time**	**B-tree time**
1,000	___ ms	___ ms	___ ms
10,000	___ ms	___ ms	___ ms
100,000	___ ms	___ ms	___ ms
1,000,000	___ ms	___ ms	___ ms

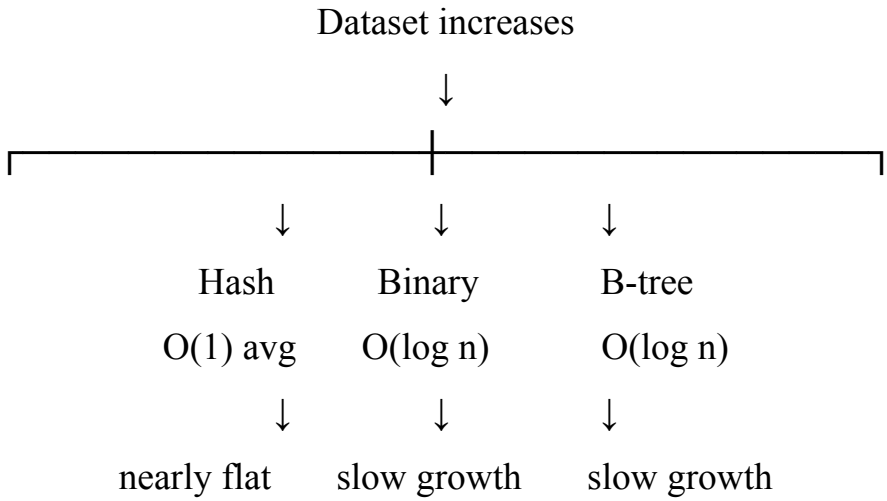
Do not invent these values. They should come from your actual implementation.

Records	**Linear Search**	**Binary Search**	**Hash Average**
1,000	1,000	~10	~1
10,000	10,000	~14	~1
100,000	100,000	~17	~1
1,000,00	1,000,000	~20	~1

0			

TASK 10 – Theory vs Experiment

The expected trend is:



Expected interpretation

If experimental results follow the theoretical prediction:

Hashing should show almost constant average lookup time, while binary search and B-tree search should increase slowly as the dataset becomes larger.

Differences between theory and actual measurements can occur because of:

- CPU/cache effects,
- memory access,
- programming language overhead,
- collision distribution,
- disk versus memory access,
- operating-system scheduling,
- implementation details.

The assignment specifically expects these theoretical/experimental differences to be interpreted rather than merely listed. Design and Analysis of Algorith…

TASK 11 – Final Design Recommendation

Recommended original design: Hash-based indexing

For the original requirement, I would recommend:

because search operations are frequent and registration numbers are unique.

Advantages

- Average search: **Θ(1)**
- Very fast repeated queries
- Simple retrieval
- Suitable for 1,000 → 1,000,000 records

Limitation

The worst case can become:

if collisions are badly distributed.

Therefore, the implementation should use a good hash function and an appropriate collision-resolution method.

TASK 12 – DAY-4 DESIGN CHALLENGE

The new requirement is:

for:

while new records are added only:

These changed requirements are explicitly given in the assignment. Design and Analysis of Algorith…

Hybrid Search Design Using a Hash Table and B-Tree

A hybrid design can use a hash table as a fast cache for frequently searched student registration numbers and a B-tree as the main index for all records.

- **Cache hit:** If the requested registration number is found in the hash table, the cached record is returned immediately.
- **Cache miss:** If the record is not present in the hash table, the system searches the B-tree. The result may then be added to the cache.
- **Update handling:** When a student record is inserted, modified, or deleted, the main B-tree index and the corresponding cached entry must be updated or invalidated.
- **Stale-record prevention:** Cache entries should be invalidated or refreshed after updates. Version numbers or timestamps can also be used to detect outdated cached records.

This approach can improve repeated searches while retaining the B-tree as the authoritative index. However, its performance depends on the cache hit rate, memory usage, and the overhead of keeping the cache consistent with the main index.

Trade-offs and Limitations

The hybrid design requires additional memory to maintain the hash-based cache and additional processing to keep cached records consistent with the B-tree. If most searches access different records, the cache hit rate may be low and the benefit may be limited. Therefore, the cache size, hit rate, and update consistency should be evaluated during testing before concluding that the hybrid design is faster than the original hash-table approach.

A. Impact Analysis

The most important change is the **very high search frequency**.

Average searches per minute:

Average searches per second:

So the system performs approximately:

on one million records.

However, records are updated only once per day.

Important observation

This means:

Read operations are extremely frequent, while write operations are relatively rare.

Therefore, spending some time building an efficient index is justified because the cost can be amortized over thousands of searches.

B. Evaluation of Original Design

Hashing remains extremely attractive:

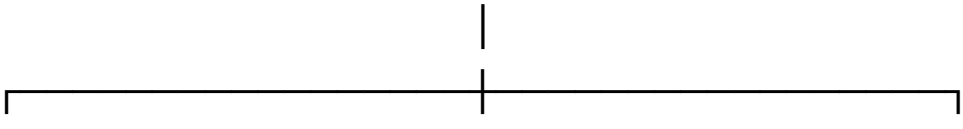
searches can be handled efficiently assuming good distribution and adequate memory.

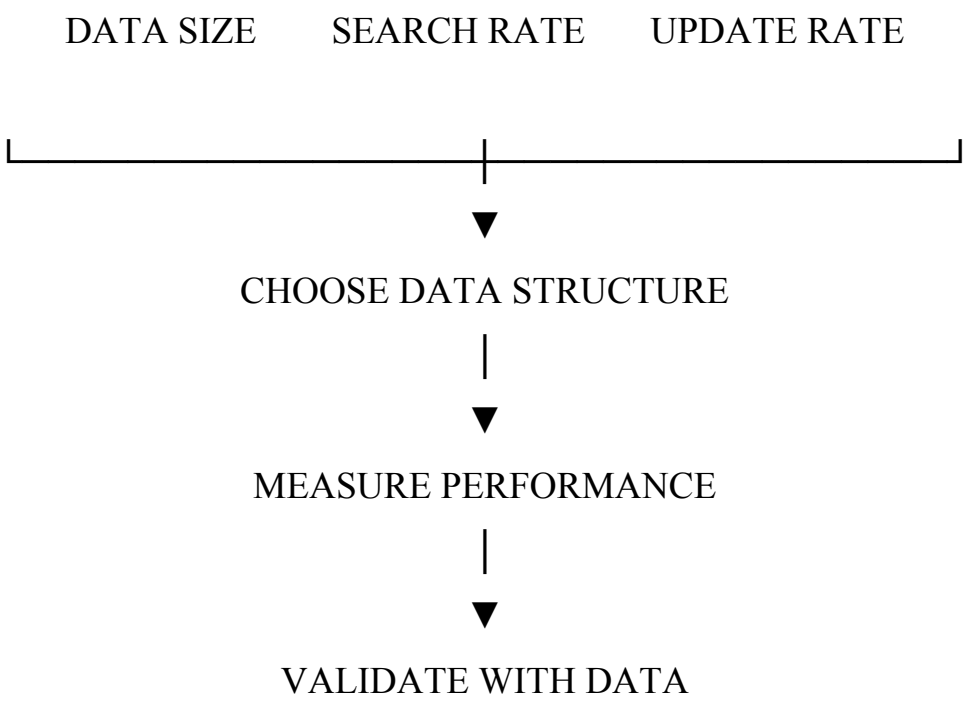
However, a production student-record database may also need:

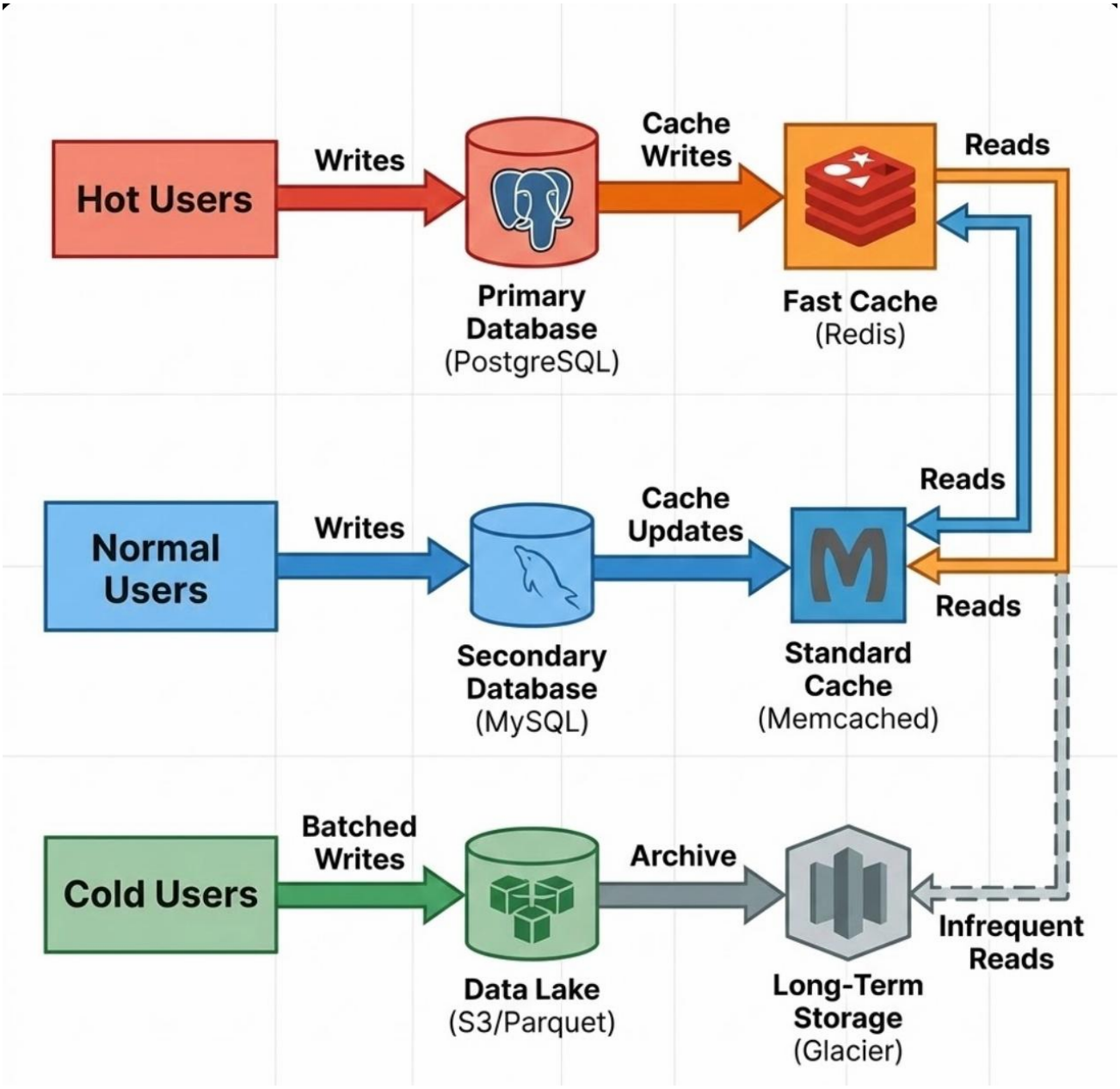
- ordered traversal,
- range queries,
- predictable worst-case performance,
- frequent retrieval,
- scalable indexing.

Therefore, an **indexed/hybrid design** can be stronger than relying on a single structure.

ALGORITHM SELECTION




DATA SIZE
SEARCH RATE
UPDATE RATE
CHOOSE DATA STRUCTURE
MEASURE PERFORMANCE
VALIDATE WITH DATA


Hot Users
Writes
Primary Database (PostgreSQL)
Cache Writes
Fast Cache (Redis)
Reads
Normal Users
Writes
Secondary Database (MySQL)
Cache Updates
Standard Cache (Memcached)
Reads
Reads
Cold Users
Batched Writes
Data Lake (S3/Parquet)
Archive
Long-Term Storage (Glacier)
Infrequent Reads